therefore current node corresponds to an intermediate character in one or more strings stored in the trie, and it can't be deleted.

Listing 6.8 shows the implementation of this method performing deletion + pruning; using figure 6.9 as a reference, you can see that, after turning it to an intermediate node (line #3), the node at the end of the path "`anti`" becomes a worthless leaf; we backtrack to its parent, remove the edge marked with '`i`' (lines #8 to #11 in listing 6.8), and then the node at the end of the path for "`ant`", which also is an intermediate node, becomes a leaf too, and can thus be removed.

When we backtrack once more, we can see that its parent is both a key node, and it does have another child, so we can't prune the tree any more.

Listing 6.8 method remove (with pruning)

```
function remove(node,
s)                                                    #1
  if s == "" then
                                                    #2
    deleted ← node.keyNode
    node.keyNode ←
false                                                    #3
    return (deleted, node.children.size == 0)
                #4
  c, tail ←
s.splitAt(0)
   #5
  if node.children[c] != null then
                          #6
```